

CIRCULAR QUEUE IMPLEMENTATIONS IN C

Circular Queue Using Arrays

```
#include <stdio.h>

#define MAX 5

int cqueue[MAX];
int front = -1;
int rear = -1;

void enqueue(int x) {
    if ((rear + 1) % MAX == front)
        printf("Queue Overflow\n");

    else {
        if (front == -1)
            front = 0;

        rear = (rear + 1) % MAX;
        cqueue[rear] = x;
    }
}

void dequeue() {
    if (front == -1)
        printf("Queue Underflow\n");

    else {
        printf("Deleted element: %d\n", cqueue[front]);

        if (front == rear)
            front = rear = -1;

        else
            front = (front + 1) % MAX;
    }
}

void peek() {
```

```
    if (front == -1)
        printf("Queue is empty\n");
    else
        printf("Front element: %d\n", cqueue[front]);
}

void display() {
    if (front == -1)
        printf("Queue is empty\n");
    else {
        int i = front;

        printf("Circular Queue elements:\n");

        while (i != rear) {
            printf("%d\n", cqueue[i]);
            i = (i + 1) % MAX;
        }

        printf("%d\n", cqueue[rear]);
    }
}

int main() {
    int choice, x;

    while (1) {

        printf("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit\n");
        scanf("%d", &choice);

        switch (choice) {

            case 1:
                printf("Enter value: ");
                scanf("%d", &x);
                enqueue(x);
                break;

            case 2:
                dequeue();
                break;
```

```
        case 3:
            peek();
            break;

        case 4:
            display();
            break;

        case 5:
            return 0;

        default:
            printf("Invalid choice\n");
    }
}
```

Circular Queue Using Array with Structure

```
#include <stdio.h>

#define MAX 5

struct CircularQueue {
    int arr[MAX];
    int front;
    int rear;
};

void init(struct CircularQueue *q) {
    q->front = -1;
    q->rear = -1;
}

void enqueue(struct CircularQueue *q, int x) {

    if ((q->rear + 1) % MAX == q->front)
        printf("Queue Overflow\n");

    else {

        if (q->front == -1)
            q->front = 0;

        q->rear = (q->rear + 1) % MAX;
        q->arr[q->rear] = x;
    }
}
```

```
void dequeue(struct CircularQueue *q) {
    if (q->front == -1)
        printf("Queue Underflow\n");
    else {
        printf("Deleted element: %d\n", q->arr[q->front]);
        if (q->front == q->rear)
            q->front = q->rear = -1;
        else
            q->front = (q->front + 1) % MAX;
    }
}

void peek(struct CircularQueue *q) {
    if (q->front == -1)
        printf("Queue is empty\n");
    else
        printf("Front element: %d\n", q->arr[q->front]);
}

void display(struct CircularQueue *q) {
    if (q->front == -1)
        printf("Queue is empty\n");
    else {
        int i = q->front;
        printf("Circular Queue elements:\n");
        while (i != q->rear) {
            printf("%d\n", q->arr[i]);
            i = (i + 1) % MAX;
        }
        printf("%d\n", q->arr[q->rear]);
    }
}

int main() {
```

```
struct CircularQueue q;
init(&q);

int choice, x;

while (1) {

    printf("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit\n");
    scanf("%d", &choice);

    switch (choice) {

    case 1:
        printf("Enter value: ");
        scanf("%d", &x);
        enqueue(&q, x);
        break;

    case 2:
        dequeue(&q);
        break;

    case 3:
        peek(&q);
        break;

    case 4:
        display(&q);
        break;

    case 5:
        return 0;

    default:
        printf("Invalid choice\n");
    }
}
```

Circular Queue Using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node *front = NULL, *rear = NULL;

void enqueue(int x) {
    struct Node* newNode =
        (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Queue Overflow\n");
    }
    else {
        newNode->data = x;

        if (front == NULL) {
            front = rear = newNode;
            rear->next = front;
        }
        else {
            rear->next = newNode;
            rear = newNode;
            rear->next = front;
        }
    }
}

void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
    }
}
```

```
    else if (front == rear) {
        printf("Deleted element: %d\n", front->data);

        free(front);
        front = rear = NULL;
    }

    else {
        struct Node* temp = front;

        printf("Deleted element: %d\n", front->data);

        front = front->next;
        rear->next = front;

        free(temp);
    }
}

void peek() {
    if (front == NULL)
        printf("Queue is empty\n");

    else
        printf("Front element: %d\n", front->data);
}

void display() {
    if (front == NULL) {
        printf("Queue is empty\n");
    }

    else {
        struct Node* temp = front;

        printf("Circular Queue elements:\n");

        do {
            printf("%d\n", temp->data);
            temp = temp->next;
        }
        while (temp != front);
    }
}
```

```
    }  
}  
  
int main() {  
  
    int choice, x;  
  
    while (1) {  
  
        printf("\n1.Enqueue 2.Dequeue 3.Peek 4.Display 5.Exit\n");  
        scanf("%d", &choice);  
  
        switch (choice) {  
  
            case 1:  
                printf("Enter value: ");  
                scanf("%d", &x);  
                enqueue(x);  
                break;  
  
            case 2:  
                dequeue();  
                break;  
  
            case 3:  
                peek();  
                break;  
  
            case 4:  
                display();  
                break;  
  
            case 5:  
                return 0;  
  
            default:  
                printf("Invalid choice\n");  
        }  
    }  
}
```